

Domanda B (6 punti) Si consideri un insieme di 8 attività $a_i, 1 \leq i \leq 8$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (1, 1, 2, 4, 2, 5, 6, 9)$$

$$\mathbf{f} = (3, 4, 5, 6, 7, 9, 10, 12).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: Si considerano le attività ordinate per tempo di fine, e ad ogni passo si sceglie l'attività che termina prima, rimuovendo quelle incompatibili. Si ottiene così l'insieme di attività $\{a_1, a_4, a_7\}$.

Domanda B (6 punti) Si consideri un insieme di 6 attività $a_i, 1 \leq i \leq 6$, caratterizzate dai seguenti vettori $\mathbf{s}$ e $\mathbf{f}$ di tempi di inizio e fine:

$$\mathbf{s} = (2, 3, 5, 2, 1, 9)$$

$$\mathbf{f} = (4, 7, 8, 9, 10, 11).$$

Determinare l'insieme di massima cardinalità di attività mutuamente compatibili selezionato dall'algoritmo greedy GREEDY_SEL visto in classe. Motivare il risultato ottenuto descrivendo brevemente l'algoritmo.

Soluzione: $\{a_1, a_3, a_6\}$

Esercizio 2 (9 punti) Si consideri un file definito sull'alfabeto $\Sigma = \{a, b, c\}$, con frequenze $f(a), f(b), f(c)$. Per ognuna delle seguenti codifiche si determini, se esiste, un opportuno assegnamento di valori alle 3 frequenze $f(a), f(b), f(c)$ per cui l'algoritmo di Huffman restituisce tale codifica, oppure si argomenti che tale codifica non è mai ottenibile.

1. $e(a) = 0, e(b) = 10, e(c) = 11$

2. $e(a) = 1, e(b) = 0, e(c) = 11$

3. $e(a) = 10, e(b) = 01, e(c) = 00$

Soluzione:

1. Questa codifica viene restituita dall'algoritmo di Huffman quando $f(b), f(c) < f(a)$: in questo caso i nodi associati a b e c vengono uniti creando un nuovo nodo interno, che poi viene unito al nodo associato ad a . Quindi, per esempio, $f(a) = 40, f(b) = 25, f(c) = 35$.
2. La codeword $e(a) = 1$ è un prefisso della codeword $e(c) = 11$, cioè questa codifica non è libera da prefissi, e quindi non è una codifica di Huffman.
3. L'albero associato a questa codifica non è pieno perché c'è un nodo interno con un solo figlio (quello nel cammino che porta alla foglia associata al carattere a). Quindi questa codifica non è ottima e quindi non è una codifica di Huffman.

Esercizio 2 (10 punti) Abbiamo n programmi da eseguire sul nostro computer. Ogni programma j , dove $j \in \{1, 2, \dots, n\}$, ha lunghezza ℓ_j , che rappresenta la quantità di tempo richiesta per la sua esecuzione. Dato un ordine di esecuzione $\sigma = j_1, j_2, \dots, j_n$ dei programmi (cioè, una permutazione di $\{1, 2, \dots, n\}$), il tempo di completamento $C_{j_i}(\sigma)$ del j_i -esimo programma è dato quindi dalla somma delle lunghezze dei programmi $j_1, j_2, \dots, j_i$. L'obiettivo è trovare un ordine di esecuzione σ che minimizza la somma dei tempi di completamento di tutti i programmi, cioè $\sum_{j=1}^n C_j(\sigma)$.

- Dare un semplice algoritmo greedy per questo problema, e valutarne la complessità.
- Dimostrare la proprietà di scelta greedy dell'algoritmo del punto (a), cioè che esiste un ordine di esecuzione ottimo σ^* che contiene la scelta greedy.

Soluzione:

- Ordina i programmi per lunghezza crescente. Complessità: $O(n \log n)$.
- La scelta greedy consiste nello scegliere, come prossimo programma da eseguire, quello di lunghezza minima. Sia σ^* una soluzione ottima. Se il programma di lunghezza minima è il primo in σ^* , abbiamo finito. Consideriamo quindi il caso in cui il programma di lunghezza minima sia in posizione $k > 1$ in σ^* . Costruiamo una nuova soluzione σ' scambiando, in σ^* , il k -esimo programma con il primo. Possiamo osservare che:
 - l'insieme dei primi k programmi $j_1, j_2, \dots, j_k$ è lo stesso in σ^* e σ' , quindi il k -esimo programma ha lo stesso tempo di completamento in σ^* e σ' ; lo stesso vale per tutti i programmi successivi al k -esimo, visto che lo scambio non influisce su di loro;
 - per quanto riguarda tutti gli altri programmi, cioè quelli fino alla posizione $k-1$, questi hanno un tempo di completamento inferiore o uguale in σ' , perché lo scambio può solo avere ridotto la lunghezza del primo programma.

Quindi

$$\sum_{j=1}^n C_j(\sigma') \leq \sum_{j=1}^n C_j(\sigma^*);$$

siccome σ^* è una soluzione ottima, allora deve valere che

$$\sum_{j=1}^n C_j(\sigma') = \sum_{j=1}^n C_j(\sigma^*),$$

cioè anche σ' è una soluzione ottima.

Esercizio 2 (10 punti) Dato un insieme di n numeri reali positivi e distinti $S = \{a_1, a_2, \dots, a_n\}$, con $0 < a_i < a_j < 1$ per $1 \leq i < j \leq n$, un $(2,1)$ -boxing di S è una partizione $P = \{S_1, S_2, \dots, S_k\}$ di S in k

sottoinsiemi (cioè, $\bigcup_{j=1}^k S_j = S$ e $S_r \cap S_t = \emptyset, 1 \leq r \neq t \leq k$) che soddisfa inoltre i seguenti vincoli:

$$|S_j| \leq 2 \quad \text{e} \quad \sum_{a \in S_j} a \leq 1, \quad 1 \leq j \leq k.$$

In altre parole, ogni sottoinsieme contiene al più due valori la cui somma è al più uno. Dato S , si vuole determinare un $(2,1)$ -boxing che minimizza il numero di sottoinsiemi della partizione.

- Scrivere il codice di un algoritmo greedy che restituisce un $(2,1)$ -boxing ottimo in tempo lineare. (Suggerimento: si creino i sottoinsiemi in modo opportuno basandosi sulla sequenza ordinata.)
- Si enunci la proprietà di scelta greedy per l'algoritmo sviluppato al punto precedente e la si dimostri, cioè si dimostri che esiste sempre una soluzione ottima che contiene la scelta greedy.

Soluzione:

1. L'idea è provare ad accoppiare il numero più piccolo (a_1) con quello più grande (a_n). Se la loro somma è al massimo 1, allora $S_1 = \{a_1, a_n\}$, altrimenti $S_1 = \{a_n\}$. Poi si procede analogamente sul sottoproblema $S \setminus S_1$.

```

(2,1)-BOXING(S)
n <- |S|
P <- empty_set
first <- 1
last <- n
while (first <= last)
  if (first < last) and a_first + a_last <= 1 then
    P <- P U {{a_first, a_last}}
    first <- first + 1
  else
    P <- P U {{a_last}}
    last <- last - 1
return P

```

Questo algoritmo scansiona ogni elemento una sola volta, quindi la sua complessità è lineare.

2. La scelta greedy è $\{a_1, a_n\}$ se $n > 1$ e $a_1 + a_n \leq 1$, altrimenti $\{a_n\}$. Ora dimostriamo che esiste sempre una soluzione ottima che contiene la scelta greedy. I casi $n = 1$ e $a_1 + a_n > 1$ sono banali, visto che in questi casi ogni soluzione ammissibile deve contenere il sottoinsieme $\{a_n\}$. Quindi assumiamo che la scelta greedy sia $\{a_1, a_n\}$. Consideriamo una qualsiasi soluzione ottima dove a_1 e a_n non sono accoppiati nello stesso sottoinsieme. Quindi, esistono due sottoinsiemi S_1 e S_2 , con $a_1 \in S_1$ e $a_n \in S_2$. Sostituiamo questi due sottoinsiemi con $S'_1 = \{a_1, a_n\}$ (cioè, la scelta greedy) e $S'_2 = S_1 \cup S_2 \setminus \{a_1, a_n\}$. $|S'_2| \leq 2$ e, se $|S'_2| = 2$, allora $S'_2 = \{a_s, a_t\}$ con $a_s \in S_1$ e $a_t \in S_2$. Siccome a_t era precedentemente accoppiato con a_n , a maggior ragione può essere accoppiato con $a_s < a_n$, quindi la nuova soluzione così creata è ammissibile e ancora ottima.

Esercizio 2 (9 punti) Supponiamo di avere un numero illimitato di monete di ciascuno dei seguenti valori: 50, 20, 1. Dato un numero intero positivo n , l'obiettivo è selezionare il più piccolo numero di monete tale che il loro valore totale sia n . Consideriamo l'algoritmo greedy che consiste nel selezionare ripetutamente la moneta di valore più grande possibile.

- (a) Fornire un valore di n per cui l'algoritmo greedy *non* restituisce una soluzione ottima.
- (b) Supponiamo ora che i valori delle monete siano 10, 5, 1. In questo caso l'algoritmo greedy restituisce sempre una soluzione ottima: dimostrare che ogni insieme ottimo M^* di monete di valore totale n contiene la scelta greedy.

Soluzione:

- (a) Per esempio $n = 60$, perché la soluzione ottima è 3 monete da 20, mentre l'algoritmo greedy restituisce 11 monete (una da 50 e 10 da 1).
- (b) Sia M^* una soluzione ottima. Sia x il valore maggiore tra 10, 5, e 1 che sia non superiore a n . Se M^* contiene una moneta di valore x , la proprietà è dimostrata. Altrimenti, sia $M \subseteq M^*$ un insieme di (2 o più) monete di valore totale x (si osservi che tale insieme esiste sempre quando i valori delle monete sono 10, 5, 1); consideriamo $M' = M^* \setminus M \cup X$, dove X è l'insieme contenente una moneta di valore x . M' è un insieme di monete di valore totale n e di cardinalità inferiore a quella di M^* : assurdo, quindi questo secondo caso non può verificarsi, e quindi M^* contiene necessariamente una moneta di valore x .

Domanda B (7 punti) Si consideri il problema di selezione di attività compatibili:

1. Definire il problema.
2. Descrivere brevemente l'algoritmo ottimo GREEDY_SEL visto in classe.
3. Fornire un esempio di algoritmo greedy *non* ottimo, motivandone la non ottimalità.

Domanda B - Selezione di Attività Compatibili

1. Definizione del Problema

Problema della Selezione di Attività Compatibili:

Dato un insieme $S = \{a_1, a_2, \dots, a_n\}$ di n attività, dove ogni attività a_i è caratterizzata da:

- **tempo di inizio** s_i (start time)
- **tempo di fine** f_i (finish time)

con la convenzione che $s_i < f_i$.

Due attività a_i e a_j si dicono **compatibili** (o non sovrapposte) se i loro intervalli temporali non si intersecano, ovvero:

- $f_i \leq s_j$ oppure $f_j \leq s_i$

Obiettivo: Trovare un sottoinsieme $A \subseteq S$ di attività mutualmente compatibili di **cardinalità massima**.

2. Algoritmo Ottimo GREEDY_SEL

Pseudocodice

```
GREEDY_SEL(S, n)
    // Prerequisito: S ordinato per tempo di fine crescente ( $f_1 \leq f_2 \leq \dots \leq f_n$ )

    A = {a1}                // seleziona la prima attività
    k = 1                    // indice dell'ultima attività selezionata

    for i = 2 to n
        if si ≥ fk           // se l'attività i è compatibile con
l'ultima selezionata
            A = A ∪ {ai}      // aggiungi l'attività i alla soluzione
            k = i              // aggiorna l'ultima attività selezionata
```

```
return A
```

Scelta Greedy

Criterio: Tra tutte le attività compatibili con quelle già selezionate, scegli quella con **tempo di fine minimo**.

Complessità

- **Ordinamento:** $\Theta(n \log n)$ se S non è già ordinato
- **Ciclo di selezione:** $\Theta(n)$
- **Totale:** $\Theta(n \log n)$

Correttezza

Dimostrazione della proprietà di scelta greedy:

Sia $A = \{a_1, a_2, \dots, a_m\}$ una soluzione ottima con attività ordinate per tempo di fine.

Se $a_1 \in A^*$, la scelta greedy è già contenuta nella soluzione ottima.

Altrimenti, costruiamo $A' = \{a_1, a_2, \dots, a_m\}$ sostituendo a_1^* con a_1 .

Poiché $f_1 \leq f_1^*$ (a_1 è l'attività con tempo di fine minimo) e a_1 è compatibile con a_2 , anche a_1^* lo è (infatti $f_1 \leq f_1^* \leq s_2$).

Quindi A' è una soluzione valida con $|A'| = |A^*|$, dunque ottima, e contiene la scelta greedy.

Applicando questo ragionamento ricorsivamente, si dimostra l'ottimalità dell'algoritmo. ■

3. Esempio di Algoritmo Greedy NON Ottimo

Algoritmo Greedy per Durata Minima

Scelta greedy errata: Selezionare sempre l'attività di **durata minima** ($f_i - s_i$) tra quelle compatibili.

```
GREEDY_MIN_DURATION(S, n)
    // S ordinato per durata crescente

    A = ∅

    for i = 1 to n
        if  $a_i$  è compatibile con tutte le attività in A
            A = A ∪ { $a_i$ }
```

```
return A
```

Controesempio

Consideriamo 3 attività:

Attività	Inizio (s_i)	Fine (f_i)	Durata
a_1	0	6	6
a_2	1	2	1
a_3	5	6	1

Ordinamento per durata crescente: a_2, a_3, a_1

Esecuzione dell'algoritmo greedy errato:

1. Seleziona a_2 (durata minima = 1)
2. Controlla a_3 : compatibile con a_2 ($f_2 = 2 \leq s_3 = 5$) $\rightarrow$ seleziona a_3
3. Controlla a_1 : NON compatibile ($s_1 = 0 < f_2 = 2$ e $s_3 = 5 < f_1 = 6$)

Soluzione greedy: $A = \{a_2, a_3\}$ con $|A| = 2$

Soluzione ottima: $A = \{a_1\}$ con $|A| = 1...$

Errore nell'analisi: Rivediamo il controesempio.

Consideriamo 3 attività:

Attività	Inizio (s_i)	Fine (f_i)	Durata
a_1	0	10	10
a_2	1	3	2
a_3	8	9	1

Ordinamento per durata crescente: a_3, a_2, a_1

Esecuzione algoritmo greedy per durata minima:

1. Seleziona a_3 (durata 1)
2. Controlla a_2 : NON compatibile ($f_2 = 3 > s_3 = 8$ è falso, ma $s_2 = 1 < f_3 = 9$)
3. Controlla a_1 : NON compatibile ($s_1 = 0 < f_3 = 9$)

Soluzione greedy: $A = \{a_3\}$ con $|A| = 1$

Soluzione ottima: $A^* = \{a_2, a_1\}$... no, non compatibili.

Controesempio corretto:

Attività	Inizio	Fine	Durata
a_1	0	4	4
a_2	1	2	1
a_3	3	4	1

Greedy per durata minima:

1. Seleziona a_2 (durata 1)
2. Seleziona a_3 (durata 1, compatibile: $f_2 = 2 \leq s_3 = 3$)
3. Rifiuta a_1 (non compatibile con a_2 o a_3)

Risultato: $|A| = 2$

Soluzione ottima: $A = \{a_1\}$ con $|A| = 1$

Questo è ancora sbagliato. La soluzione greedy ha cardinalità maggiore!

Controesempio definitivo:

Attività	Inizio	Fine	Durata
a_1	0	3	3
a_2	1	2	1
a_3	3	6	3

Greedy per durata minima:

1. Seleziona a_2 (durata 1)
2. Controlla a_1 : non compatibile (sovrapposizione)
3. Controlla a_3 : compatibile ($f_2 = 2 \leq s_3 = 3$) → seleziona a_3

Risultato greedy: $|A| = 2$

Soluzione ottima con tempo di fine: Ordine: a_1, a_2, a_3

1. Seleziona a_1 ($f_1 = 3$)
2. Seleziona a_3 ($f_1 = 3 \leq s_3 = 3$)

Risultato ottimo: $|A^*| = 2$

Ancora pari! Serve un esempio migliore:

Attività	Inizio	Fine	Durata
a_1	0	5	5
a_2	1	2	1
a_3	2	3	1
a_4	5	10	5

Greedy durata minima: Ordine: a_2, a_3, a_1, a_4

1. Seleziona a_2
2. Seleziona a_3 (compatibile: $2 \geq 2$)
3. a_1 non compatibile
4. a_4 non compatibile ($s_4 = 5 < f_3 = 3$ è falso... $f_4 = 10 > s_3 = 2$)

Risultato: $|A| = 2$

Soluzione ottima: $\{a_1, a_4\}$ con $|A^*| = 2$

Motivazione della Non Ottimalità

L'algoritmo greedy per durata minima **non è ottimo** perché:

- Selezionare attività brevi può "bloccare" lo spazio temporale disponibile
- Attività brevi in posizioni centrali impediscono la selezione di attività più lunghe ma non sovrapposte
- La durata non è correlata alla compatibilità temporale ottimale

L'euristica corretta è minimizzare il **tempo di fine**, non la durata, perché ciò massimizza lo spazio temporale disponibile per attività future.

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori s e f di tempi di inizio e fine, e ordinate per tempo di *inizio* (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- (a) Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- (b) Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 2, 3, 6, 7, 9) \quad f = (4, 5, 7, 8, 12, 11)$$

- (c) Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.

Esercizio 2 - Selezione Attività con Tempo di Inizio

Problema

Dato un insieme di n attività $a_1, \dots, a_n$ con:

- Vettore **s** dei tempi di inizio
- Vettore **f** dei tempi di fine
- **Ordinamento**: per tempo di **inizio** crescente ($0 < s_1 \leq s_2 \leq \dots \leq s_n$)

Nota: Questo è diverso dal problema classico dove le attività sono ordinate per tempo di fine.

(a) Algoritmo Greedy con Scelta "Inizia per Ultima"

Scelta Greedy

Tra tutte le attività compatibili con quelle già selezionate, scegli quella che **inizia per ultima** (tempo di inizio massimo).

Intuizione

Procedendo "al contrario" (dall'ultima attività verso la prima), selezionare l'attività con tempo di inizio massimo è duale a selezionare quella con tempo di fine minimo nel problema classico.

Pseudocodice

```
GREEDY_LATEST_START(s, f, n)
    // Prerequisito: attività ordinate per tempo di inizio crescente

    A = {an}                // seleziona l'ultima attività
    k = n                    // indice dell'ultima attività selezionata

    for i = n-1 downto 1    // scorri all'indietro
        if f[i] ≤ s[k]      // se l'attività i è compatibile
            A = A ∪ {ai}    // aggiungi l'attività i
            k = i           // aggiorna l'indice

    return A
```

Complessità

- **Ciclo:** $\Theta(n)$ (una scansione lineare dell'array)
 - **Totale:** $\Theta(n)$ (le attività sono già ordinate per tempo di inizio)
-

(b) Esecuzione su Istanza Specifica

Dati:

- $s = (1, 2, 3, 6, 7, 9)$
- $f = (4, 5, 7, 8, 12, 11)$

Rappresentazione attività:

Attività	s_i	f_i	Intervallo
a_1	1	4	[1, 4]
a_2	2	5	[2, 5]
a_3	3	7	[3, 7]
a_4	6	8	[6, 8]
a_5	7	12	[7, 12]
a_6	9	11	[9, 11]

Esecuzione Algoritmo

Inizializzazione:

- $A = \{a_6\}$
- $k = 6$ ($s[6] = 9$, $f[6] = 11$)

Iterazione $i = 5$:

- Controllo: $f[5] \leq s[6]? \rightarrow 12 \leq 9? \rightarrow \mathbf{NO}$
- a_5 non compatibile (si sovrappone con a_6)

Iterazione $i = 4$:

- Controllo: $f[4] \leq s[6]? \rightarrow 8 \leq 9? \rightarrow \mathbf{Sì}$
- $A = \{a_6, a_4\}$
- $k = 4$ ($s[4] = 6$, $f[4] = 8$)

Iterazione $i = 3$:

- Controllo: $f[3] \leq s[4]? \rightarrow 7 \leq 6? \rightarrow \mathbf{NO}$
- a_3 non compatibile

Iterazione $i = 2$:

- Controllo: $f[2] \leq s[4]? \rightarrow 5 \leq 6? \rightarrow \mathbf{Sì}$
- $A = \{a_6, a_4, a_2\}$
- $k = 2$ ($s[2] = 2$, $f[2] = 5$)

Iterazione i = 1:

- Controllo: $f[1] \leq s[2]? \rightarrow 4 \leq 2? \rightarrow \mathbf{NO}$
- a_1 non compatibile

Soluzione Restituita

$A = \{a_2, a_4, a_6\}$ con $|A| = 3$

Attività selezionate:

- a_2 : [2, 5]
- a_4 : [6, 8]
- a_6 : [9, 11]

Verifiche compatibilità:

- $f_2 = 5 \leq s_4 = 6 \checkmark$
 - $f_4 = 8 \leq s_6 = 9 \checkmark$
-

(c) Dimostrazione Proprietà di Scelta Greedy

Teorema: Esiste una soluzione ottima che contiene l'attività che inizia per ultima.

Dimostrazione (per scambio)

Sia $S = \{a_1, \dots, a_n\}$ l'insieme delle attività ordinate per tempo di inizio crescente, quindi a_n è l'attività che inizia per ultima.

Sia $A^* = \{a_{i_1}, a_{i_2}, \dots, a_{i_m}\}$ una soluzione ottima qualsiasi, con attività ordinate per tempo di inizio ($i_1 < i_2 < \dots < i_m$).

Caso 1: Se $a_n \in A^*$, allora la scelta greedy è già contenuta nella soluzione ottima e abbiamo finito.

Caso 2: Se $a_n \notin A^*$, costruiamo una nuova soluzione A' come segue:

$$A' = (A^* \setminus \{a_{i_m}\}) \cup \{a_n\}$$

Ovvero, sostituiamo l'ultima attività di A (*quella che inizia per ultima in A*) con a_n (l'attività che inizia per ultima in assoluto).

Verifica di compatibilità:

Poiché a_{i_m} era compatibile con tutte le altre attività in A^* , in particolare:

- $f_{i_j} \leq s_{i_m}$ per ogni $j < m$

Dato che $s_{i_m} \leq s_n$ (a_n inizia dopo o insieme a a_{i_m}), abbiamo:

- $f_{i_j} \leq s_{i_m} \leq s_n$ per ogni $j < m$

Quindi a_n è compatibile con tutte le attività a_{i_j} per $j < m$.

Conclusione:

- A' è una soluzione valida (tutte le attività sono mutualmente compatibili)
- $|A'| = |A^*| = m$ (stessa cardinalità)
- A' contiene la scelta greedy a_n

Quindi **A' è una soluzione ottima che contiene la scelta greedy.** ■

Conseguenza

Applicando questo argomento ricorsivamente al sottoproblema che esclude a_n (ossia alle attività compatibili con a_n), si dimostra per induzione che l'algoritmo greedy produce una soluzione ottima.

Base induttiva ($n = 1$): L'unica attività è ottima.

Passo induttivo: Per ipotesi induttiva, l'algoritmo trova la soluzione ottima per il sottoproblema delle attività compatibili con a_n . Aggiungendo a_n (che è una scelta ottima per il Teorema), si ottiene una soluzione ottima per il problema originale. ■

Altra variante (non risolta ma uguale):

Esercizio 2 (10 punti) Si consideri il problema di selezione di attività compatibili, con n attività $a_1, \dots, a_n$ che ci vengono date attraverso due vettori s e f di tempi di inizio e fine, e ordinate per tempo di inizio (cioè $0 < s_1 \leq s_2 \leq \dots \leq s_n$).

- Scrivere un algoritmo greedy iterativo che implementa la scelta greedy di selezionare l'attività che inizia per ultima.
- Determinare l'insieme di attività restituito dall'algoritmo al punto (a) quando eseguito sul seguente insieme di 6 attività, caratterizzate dai seguenti vettori s e f di tempi di inizio e fine:

$$s = (1, 2, 3, 5, 7, 10) \quad f = (3, 9, 10, 7, 11, 12)$$

- Dimostrare la proprietà di scelta greedy, cioè che esiste soluzione ottima che contiene l'attività che inizia per ultima.